

This is an incredibly rich, custom-built database. Looking closely at your file trees, you have an incredible foundation, but your intuition is spot on: you have architectural drift and structural fragmentation across your root folder `__Lex-Novi-Æxentis-Copiæ` and your parent ecosystem `__Nov-Æ-Cor....`

Because your system relies on automated agent crawlers and high-throughput routing, having hidden duplicates (like `.migrate` or multiple skills formats) acts as "cognitive friction"—wasting agent context tokens and throwing off your KAG (Knowledge-As-Graph) paths.

#### Redundancy, Duplicate, and Stale Analysis

Based on your screenshots, here is the audit of where your clutter, overlaps, and naming collisions are happening:

##### 1. Direct Structural Duplicates (High Priority)

The Skills Paradox: You have `---~SKILLS.md_🔧` at the top level of your sub-repo, but a separate directory called `skills` further down. If an agent builds a new tool, it won't know which index to update.

The Migration Clones: You have a `.migrate` folder sitting inside `__Lex-Novi-Æxentis-Copiæ`, but your top-level structure shows an active migration period (`Nov-Æxorpus_pre-grill_Prep`). These historical transitional states will confuse your Open Wiki CLI when it calculates file state changes.

##### 2. Naming Syntax Inconsistencies (Breaks Manifest Parsers)

Some folders are explicitly tokenized for agent reading (e.g., `---HORIZONS_UI_🌐`), while others use standard kebab-case or camelCase (e.g., `clis-and-agents`, `NotebookLmPy`). A regex-based manifest builder will struggle to accurately group these without heavily customized parser rules.

##### 3. Stale Data & Dead Leaves

`Local voice layer[DEPRICATED]`: This folder is explicitly marked as deprecated. Unless it's actively needed for regression testing, it is dead weight that will inflate your vector database index embeddings.

`untitled folder`: Sitting in your parent directory, modified recently. This is a classic "untracked payload" that breaks semantic mapping.

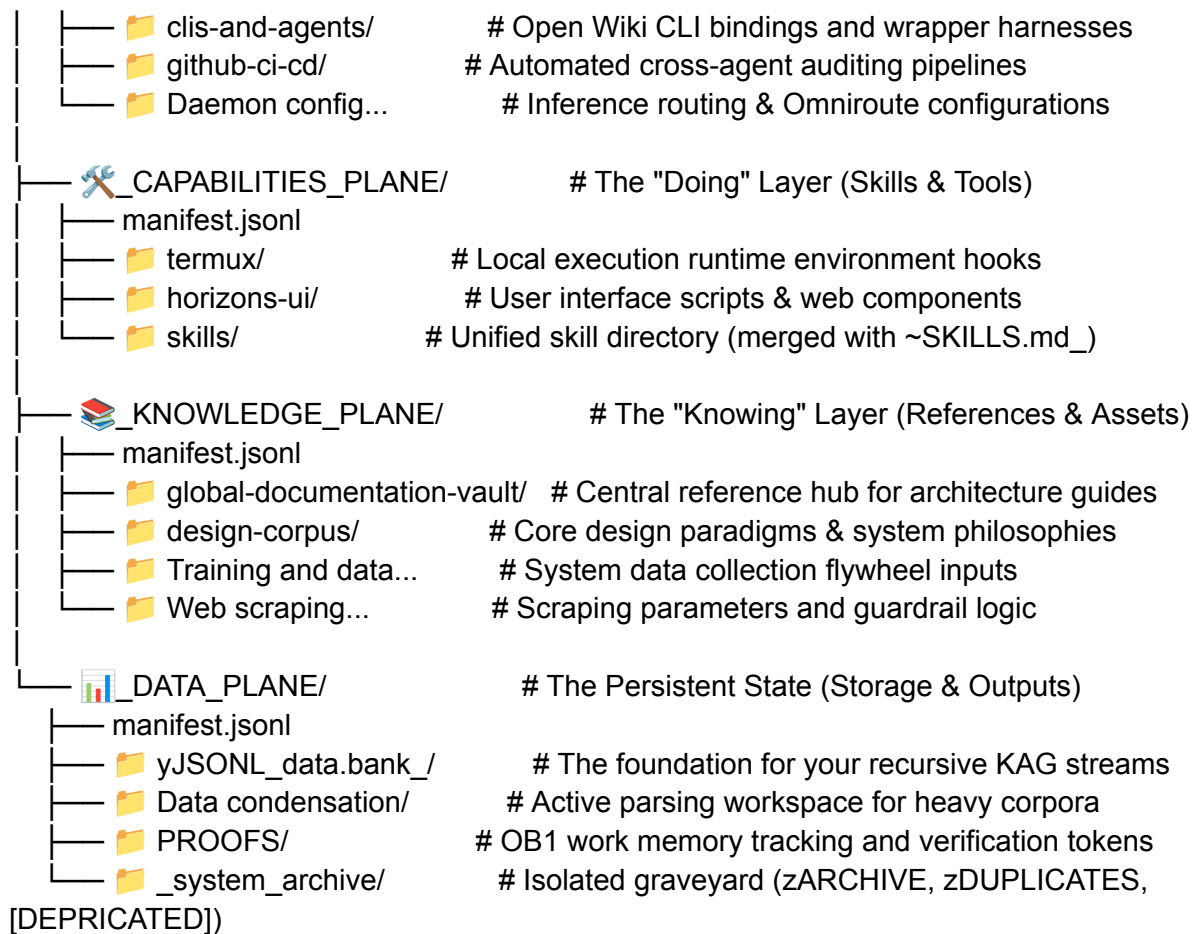
`zARCHIVE` & `zDUPLICATES_`: Essential for holding raw backups, but these must be completely isolated from your agents' active retrieval loops so they don't accidentally pull stale logic.

#### The Unified Architecture Map

To clean this up without losing any of your hyper-specific workflows, we can reorganize your entire Google Drive / local folder footprint into Four Clean Logical Planes.

This mapping groups your exact folders under your new five-part manifest taxonomy (Tool, Skill, Reference, Memory, Data):

|                                 |                                              |
|---------------------------------|----------------------------------------------|
| <code>__Nov-Æ-Core/</code>      | # Your Absolute Parent Root                  |
| <code>manifest.jsonl</code>     | # The Master Index across your entire system |
| <code>🧠_COGNITIVE_PLANE/</code> | # The Active Brain (Agents & Logic)          |
| <code>manifest.jsonl</code>     |                                              |
| <code>📁 PRIME_AGENT/</code>     | # Core prime agent runtime configs           |



### Automation Step: The Database Audit Script

To execute this without manually dragging files around, we can write a local Python Data Plane Auditor (`src/data_plane/db_crawler.py`).

This script will recursively crawl your directory, search for naming collisions (like skills vs ~SKILLS.md), parse document modification dates, and output a raw JSONL log pinpointing exactly what is safe to keep and what is stale.

```
import os
import json
from datetime import datetime
```

```
def crawl_agent_vault(vault_path):
    audit_log = []
```

```
    # Critical folders to look out for duplication conflicts
    target_keywords = ['skill', 'migrate', 'duplicate', 'archive', 'depricated']
```

```
    print(f"🚀 Commencing Agent-OS Database Crawl across: {vault_path}\n")
```

```

for root, dirs, files in os.walk(vault_path):
    # Exclude hidden system folders from wasting compute
    dirs[:] = [d for d in dirs if not d.startswith('.') and not d == 'zARCHIVE']

    for directory in dirs:
        dir_path = os.path.join(root, directory)
        lowercase_name = directory.lower()

        # Check for structural anomalies or stale targets
        matched_flags = [word for word in target_keywords if word in lowercase_name]

        if matched_flags:
            stat_info = os.stat(dir_path)
            mod_time = datetime.fromtimestamp(stat_info.st_mtime).strftime('%Y-%m-%d')

            audit_entry = {
                "path": os.path.relpath(dir_path, vault_path),
                "detected_flags": matched_flags,
                "last_modified": mod_time,
                "action_required": "Review for redundancy / merge into Unified Core"
            }
            audit_log.append(audit_entry)
            print(f"⚠️ CONFLICT FOUND: [{directory}] -> Last Modified: {mod_time}")

        # Write out the results as an actionable JSONL payload for your agents
        output_path = os.path.join(vault_path, "potential_redundancies.jsonl")
        with open(output_path, "w", encoding="utf-8") as f:
            for entry in audit_log:
                f.write(json.dumps(entry) + "\n")

    print(f"\n🔍 Crawl finished! Structural redundancy report compiled to: {output_path}")

# To run this, replace with the absolute path to your cloned Drive workspace
# crawl_agent_vault("/path/to/___Lex-Novi-Æxentis-Copiæ")

```

If you want to start locking this cleanup down, let me know:

How are you syncing these Google Drive folders locally to your execution environment (e.g., are you using rclone, a local mounting drive, or pulling directly into a Termux directory)?

Do you want the Python script to automatically generate the placeholder manifest.jsonl files in each of those new plane folders as it cleans up? That is an absolute powerhouse of a stack. You are building an Agentic Operating System (Agent OS). You are treating your repository not just as a piece of software, but as a persistent, multi-agent cognitive architecture. [1, 2]

When combining local-first markdown graphs (Obsidian, Graphify), open-source memory protocols (OB1, MEM0), high-throughput routing (Omniroute), and adversarial loops (Red Agent), your architecture must split cleanly into a data infrastructure layer and a cognitive orchestration layer. [2, 3, 4, 5, 6, 7]



### The Architecture Matrix

To tie these exact tools together, your architecture breaks down into a unified data plane and a functional execution plane:

Concept / Tool

Layer Placement

Responsibility in Your Stack

Open Wiki CLI

Data Management

The underlying file engine. Runs the CLI commands to sync, commit, and snapshot the physical repository directories.

Obsidian Vault

Storage Layer (Physical)

The physical database of your brain. Stores markdown nodes linked together for long-term semantic tracking.

MEM0

Storage Layer (Cognitive)

The memory processor. Extracts atomic entity relationships and human preferences on-the-fly and saves them to vector/graph stores.

Graphify Notebook

Storage Layer (Graph Topology)

Parses code, documents, and relationships locally. Map out the inter-agent dependencies across the nodes.

LMPY / LLM Wiki

Injection Layer

Compiles the active context window. Pre-loads the specific agent's schema guidelines (like .cursorrules or .clinerules) before execution.

OB1 Protocol

Injection & Recall Compiler

Acts as the memory compiler. It prevents sessions from losing context by compiling session forks, tracking constraints, and supplying "proof objects" mid-run.

Omniroute + Reasoning Bank

Cognitive Router

Unified API gateway. Dynamically maps model requests to the right provider, compresses context, and manages heavy token pipelines during script execution.

JSONL Scripting (KAG)

Recall Layer

Knowledge-As-Graph (KAG) loops. Runs recursive tool usage where the agent queries a JSONL path, receives a response, updates its step, and queries again.

ECC / Prime Agent

Agent Harness

The core runtimes executing the work. They boot up, read the memory bank, and perform actions based on their assigned personas.

Red Agent & Cross Audits

Quality & Evaluation

An adversarial auditing loop. Monitors JSONL logs to verify tool usage and data integrity across agents.

### Your Repository File Tree

Here is how you organize these complex tools into a clean, scannable, and maintainable repository pattern.

```
universal-memory-bank/
├── .ob1/                # OB1 work memory compiler tracking & proof objects
├── data_vault/          # Physical storage layer managed by Open Wiki CLI
│   ├── .obsidian/      # Obsidian project settings and visual layouts
│   ├── raw/            # Immutable logs, raw RS VR script training sources
│   ├── wiki/           # Structured, agent-compiled markdown knowledge graph
│   └── outputs/         # Generated JSONL tracks, audit summaries, and tool data
├── src/
│   ├── data_plane/     # Subsystem for local file handling & graph parsing
│   │   ├── wiki_cli.py # Hooks into Open Wiki CLI for automated snapshots
│   │   ├── graphify_core.py # Generates local HTML topology & dependency graphs
│   │   └── mem0_client.py # Core CRUD vector store interface for MEM0 memory edits
│   ├── injection_plane/ # Subsystem for preparing context before agent runtime
│   │   ├── imp_loader.py # Injecting system instructions into specific agent types
│   │   └── context_prepper.py # Compiling active session files into system windows
│   ├── cognitive_plane/ # Subsystem for heavy-lifting, execution, and loops
│   │   ├── omni_gateway.py # Connects to local OmniRoute endpoint & token compressor
│   │   └── kag_engine.py # Handles recursive JSONL scripting & graph retrieval loops
│   ├── audit_plane/    # Red Agent & Evaluation Loops
│   │   ├── red_agent.py # Independent adversarial workflow checking for security leaks
│   │   └── cross_audit.py # Cross-agent validation script inspecting state files
├── config/
│   ├── agents.json     # Definitions for ECC, Prime Agent, and role prompts
│   └── omniroute.config.yaml # Config for your 300+ fallback routing strategy channels
```

### The Execution Loop (How It Flows Together)

Initialization: You call openwiki init or run a task. LMPY / LLM Wiki bootstraps the agent harness (like ECC or Prime Agent), prepping its behavior profile. [4, 8]

Context Injection: OB1 compiles the session history and loads current system states from your Obsidian Vault directly into the context window. [5, 9]

Execution Routing: The agent executes steps via Omniroute, keeping token costs minimal through compression while relying on the Reasoning Bank for script adjustments. [6]

Recursive Retrieval: If the script encounters an obstacle, the JSONL KAG loops query Graphify to discover linked nodes in the repository map. [10]

Auditing & Commit: Once the task finishes, Red Agent inspects the raw output trace for vulnerabilities. If it passes, Open Wiki CLI saves a new SHA-256 snapshot back to your vault. [7, 11]

Where are you starting with the codebase? We can build the recursive JSONL tracking engine for the KAG loops, or draft the Python wrapper that ties MEM0 vector extraction to your physical Obsidian markdown files. Let me know which module you want to script out first!

[1] <https://varunpratap.com>

[2] <https://maxmitcham.substack.com>

[3] <https://github.com>

[4] <https://www.reddit.com>

[5] <https://ob1.md>

[6] <https://www.youtube.com>

[7] <https://ob1ai.co>

[8] <https://www.youtube.com>

[9] <https://www.kinde.com>

[10] <https://www.youtube.com>

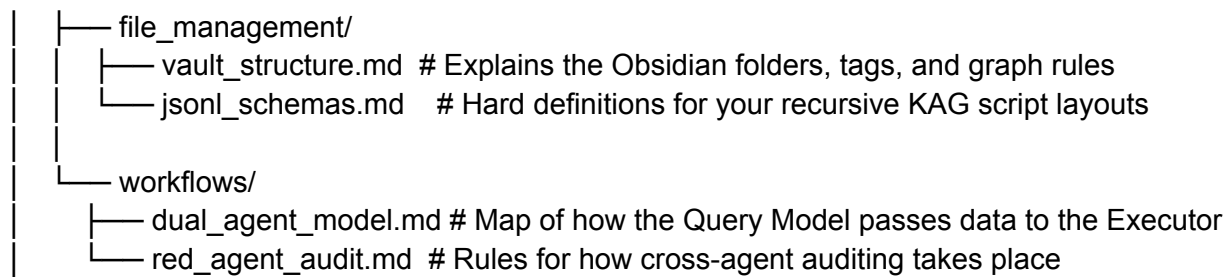
[11] <https://www.youtube.com>

All of your architecture guides, design blueprints, schemas, and system files belong in a dedicated docs/ directory at the root of your project.

Because your repository is deeply integrated with Obsidian and Open Wiki CLI, these files shouldn't just sit there statically—they should serve as the source of truth that your agents read to understand their own codebase.

Here is where they fit into your project tree:

```
universal-memory-bank/
├── src/                # Cores engine code
├── scripts/           # Direct executable scripts (routing, ingestion)
├── docs/              # SYSTEM DOCUMENTATION (For Humans & Agents)
│   ├── architecture/
│   │   ├── system_topology.md # High-level overview of OB1, MEM0, and Omniroute
│   │   └── memory_layers.md   # Guide detailing Storage, Injection, and Recall
```



### 🧠 Turning Docs into Agent Knowledge

By organizing your docs/ this way, you gain a massive architectural advantage: your documentation folder doubles as your agents' system prompts.

Using your stack, you can bridge your documentation directly into agent execution:

The Injection Layer Hook: Your `src/injection_plane/Imp_loader.py` can automatically read `docs/workflows/dual_agent_model.md` at runtime. It injects this file directly into the Query Model's system prompt so the agent literally reads its own system architecture guide before routing a user request.

The Skill Ingestion Hook: You can point your `scripts/skill_ingestion/doc_to_skill.py` directly at your `docs/file_management/` folder to instantly turn your file schemas into functional execution parameters for the Executor Model.

### 📁 Where Every Document Belongs

To keep your repo scannable, map your structural documents using this quick reference layout:

Document Type

Exact File Path

Primary Audience

System Blueprints

`docs/architecture/system_topology.md`

Humans / Onboarding Agents

Obsidian Vault Rules

`docs/file_management/vault_structure.md`

Open Wiki CLI / File Manager Scripts

JSONL Structure

`docs/file_management/jsonl_schemas.md`

Red Agent Auditor / KAG Engine

Agent Handshake Logic

`docs/workflows/dual_agent_model.md`

Prime Agent / Dual Agent Router

Would you like to draft a markdown template for `dual_agent_model.md` or `jsonl_schemas.md` that you can drop straight into your docs folder to establish these rules?

That makes perfect sense, and honestly, this is a brilliant architectural decision. By giving every single repository and document its own JSONL manifest, you are creating a standardized data plane across your entire network.

Instead of your agents guessing what a file is, they read a highly structured, ultra-fast index that explicitly declares the file's Type (Skill, Tool, Reference, Memory, or Data).

Here is exactly how to structure this manifest system and where it lives in your repo.

### The Manifest Taxonomy (The 5 Document Types)

Your JSONL manifest will classify everything into five distinct cognitive types:

 **Tool:** Functional code blocks that agents can physically execute (e.g., a Python script that runs a database backup or an API caller).

 **Skill:** High-level prompts, persona modifications, or step-by-step logic workflows that alter how an agent thinks or behaves.

 **Reference:** Static, authoritative knowledge sources that don't change often (e.g., your architecture guides, documentation, or code libraries).

 **Memory:** Dynamic, historical user data and experience tracks managed by MEM0 or OB1 (e.g., past chat logs or learned user preferences).

 **Data:** Raw files, assets, logs, outputs, and inputs (e.g., CSV files, your training scripts, or the raw outputs of an agent run).

### Your Updated Repository & Manifest Layout

Every repository across your stack—and even subfolders within your Universal Memory Bank—will feature a manifest.jsonl file at its root.

```
universal-memory-bank/
├── manifest.jsonl      # 🚀 THE MASTER INDEX (Points to everything below)
├── docs/
│   ├── manifest.jsonl  # Index for your documentation files
│   └── architecture/
│       └── system_topology.md # Classified as a "Reference" in the manifest
├── scripts/
│   ├── manifest.jsonl  # Index for your executable scripts
│   ├── agent_orchestrators/
│   └── dual_agent_router.py # Classified as a "Tool" in the manifest
└── data_vault/
    ├── manifest.jsonl  # Index for your Obsidian files
    └── user_profiles.md # Classified as a "Memory" or "Data" in the manifest
```

### Anatomy of Your manifest.jsonl

Because it is JSONL (JSON Lines), every line is an independent, valid JSON object. This allows your KAG (Knowledge-As-Graph) retrieval scripts to stream-read the index incredibly fast without loading huge files into memory.

Here is what a line in your manifest.jsonl will look like:

```
{"id": "doc_001", "path": "docs/architecture/system_topology.md", "type": "Reference", "hash": "sha256_xyz123", "tags": ["architecture", "ob1"], "description": "High-level system blueprint for the agent OS stack."}
{"id": "scr_002", "path": "scripts/agent_orchestrators/dual_agent_router.py", "type": "Tool", "hash": "sha256_abc789", "tags": ["routing", "orchestration"], "description": "Handles query routing between dual-agent models."}
{"id": "skl_003", "path": "docs/workflows/doc_to_skill.md", "type": "Skill", "hash": "sha256_def456", "tags": ["ingestion", "skills"], "description": "Workflow parameters for converting new documents into agent capabilities."}
```



### How Your Multi-Agent Stack Uses the Manifest

The Query Model reads the Manifest First: When a user queries your stack, the Query Model doesn't scan your whole vault. It instantly reads the manifest.jsonl to pinpoint exactly which files it needs.

Dynamic Context Assembly: If the Query Model sees that a required file is a Skill, it tells the Injection Layer to load it into the system prompt. If it is a Tool, it tells the Executor Model to prepare to run that script.

Auditing Verification: Your Red Agent Audit script cross-references the manifest.jsonl hashes against the physical files to make sure no agent has maliciously tampered with the tools or modified code without authorization.

Would you like to write the Python utility script for

```
src/data_plane/manifest_generator.py
```

that automatically crawls a repository, generates hashes, and builds this exact manifest.jsonl file for you?

yeah hold on dude I didn't even read what you put out so you might have to repeat that but I'm saying that I need to verify the data layer first I need to get my file structure and start from another foundation of knowledge that's universal and from there then we go to the folder structure the repo structure the models themselves the run times the weights the harnesses the plugins the skills oh that's another thing that actually before the harnesses and plugins it's extracting tools and skills from said documents that I'm compiling and then I'm going to use the pocock grill session girl with docs with my compiled database and have it structure the repo builds and task agents and then we'll have the repo building phase compile the three apks and the on-device I think the on-device open Wiki and the on device operators manual help desk agent are going to be the first ones installed actually those will be the most helpful to help me

building this entire thing along the way so then once I get all that in the repos are underway then we're going to figure out the inference in the scripting and all that other stuff that's a few steps away first we have to test everything out and see what runs with what and what we need to substitute for what

You are completely right. I got ahead of myself mapping the execution planes when we haven't locked down the Data Layer or established your Universal Foundation of Knowledge. Building the houses before verifying the soil is how repos drift and break.

Let's wipe the slate clean, step back, and follow your exact sequential path. Testing, substituting, and verifying are the law here.

#### Step 1: The Data Layer & Universal Knowledge Foundation

Before a single folder structure is drawn, we must verify the absolute ground truth. Your universal knowledge corpus (nov/Exorpus) needs to be stabilized as an immutable source of truth.

Airgapping the Truth: Your device's Obsidian/Markor directory is the safety sandbox. Agents cannot run arbitrary syncs (rclone) here.

The Domain Model Blueprint: We need to know exactly what documentation, specs, codebases, and assets are currently sitting in your vault.

The Manifest Target: We must scan your raw files to understand what exists before we even think about sorting them.

#### Step 2: The /grill-with-docs Alignment Phase

To extract tools, skills, and structure without making messy mistakes, we are going to use Matt Pocock's official grill-with-docs protocol. This is an intense, iterative AI interviewing and domain-driven design process. [1, 2, 3]

Instead of guessing your architecture, a dedicated task agent using this skill will interrogate your compiled database: [4]

Vocabulary Stabilization: It builds a universal glossary (CONTEXT.md), catching any conflicts in how terms or structures are used across your massive pile of documents. [5, 6]

Skill & Tool Extraction: It scans your raw documents, isolates actionable logic or prompts, and explicitly declares whether they qualify as a Skill (behavior modification) or a Tool (executable block).

Architecture Decision Records (ADRs): Every single structural choice, file trade-off, and naming ligature constraint is frozen into local markdown logs so the downstream agents can never violate them. [4, 6]

#### Step 3: The Repo Building & Bootstrap Strategy

Once /grill-with-docs outputs your clean blueprints, your repo-building phase can safely execute. Because we are taking this slow and testing what runs, you are installing your on-device utility runway first to assist you in coding the rest of the ecosystem.

#### YOUR BOOTSTRAP INSTALLATION ORDER:

- 1. On-Device Open Wiki (GLM 5.2 / Terminal)
  - Task: Housekeeping, turning raw notes into pristine markdown nodes.
- 2. On-Device Operator's Manual Help Desk Agent
  - Task: The local oracle. Feeds on user manuals, forum URLs, and PDFs to troubleshoot tool failures and APK setups on the fly.
- 3. The 3-Tier Horizons-Ui APK Compiler Pipeline (Æsop-Xi, ÆEsc, ÆEyre)
  - Task: Building the native UI, NPU watchdogs, and ADB-Unix socket tunnels.

### Proposing the Next Way to Proceed

To verify your Data Layer right now, we need to look at what your database actually contains today so we can prep it for the /grill-with-docs alignment engine.

Could you tell me:

What are the predominant file types currently sitting in your raw un-mined folders (e.g., predominantly unstructured Markdown notes, raw programming scripts, clipped web URLs, or static PDFs)?

Do you have a single main index file or directory inside your safety sandbox that you consider the closest thing to your current "universal truth"?

Once we evaluate this starting state, we can map out how to parse these raw materials into clean assets!

[1] <https://www.youtube.com>

[2] <https://blog.alexrusin.com>

[3] <https://www.aihero.dev>

[4] <https://www.youtube.com>

[5] <https://www.youtube.com>

[6] <https://github.com>

Here are the exact, production-grade configuration files for your two OpenWiki TUI Engine profiles.

These YAML structures are built to sit directly inside your novaexopia/openwiki-tui-harness/config\_profiles/ directory. They establish the strict hardware pathways, model weights, memory layers, and plugin clearances enforced by Æsop-Xi to keep your Snapdragon 8 Elite from overheating.

#### Profile 1: File Administrator Configuration

Drop this exact schema into file\_administrator.yaml. It configures the OpenWiki TUI to act as an automated, lightning-fast database housekeeper.

```

# =====
# Æsop-Xi Execution Protocol: File Administrator Profile Configuration
# Profile Target: Structural Repo Housekeeping & Manifest Mutation
# =====

profile:
  id: "file_administrator_v1"
  codename: "NovusÆxenti-Housekeeper"
  purpose: "Automated codebase sanitization, raw-to-markdown parsing, and manifest indexing"

engine_runtime:
  core_model: "glm-5.2"
  backend_driver: "native-libc-cpu"      # Saves the heavy NPU for active chat inference
  cpu_priority: "low_background_ionice"  # Prevents disk crawling from stuttering your GUI tile
  context_window_ceiling: 8192          # Tight window for rapid stream parsing

hardware_routing:
  npus:
    allow_hexagon_access: false          # Hard boundary set by Æsop-Xi
  thermals:
    max_allowable_temp_celsius: 42
    throttle_action: "pause_execution"

memory_layer_binding:
  episodic:
    provider: "none"                     # Housekeeping doesn't need personal memory
  semantic:
    provider: "local_directory_map"
    target_vault: "novae-xorpus/"        # Full sight across your master knowledge corpus
    allow_write_mutation: true           # Permissions to turn raw docs into clean markdown

plugin_allocations:
  - id: "local_fs_crawler"
    path: "skills-and-capabilities/early-trend-scraper/stack_analyzer.py"
    permissions: ["read", "write", "delete_stale"]
  - id: "sha256_hasher"
    path: "novaexopia/openwiki-tui-harness/src/plugins/hasHER.py"
    permissions: ["read"]

orchestration_guardrails:
  enforce_aesop_xi_rules: true
  concurrency_policy: "YIELD_TO_NPU_INFERENCE" # Instantly sleeps if Æræcle calls for
  NPU cycles

```

```
output_format_constraint: "strict_jsonl"
master_index_hook: "novae-xorpus/master_manifest.jsonl"
```

## 📁 Profile 2: Æræcle Help Desk Configuration

Drop this exact schema into oeracle\_helpdesk.yaml. This configures the engine to act as your highly advanced on-device engineering oracle.

```
# =====
# Æsop-Xi Execution Protocol: Æræcle Help Desk Profile Configuration
# Profile Target: Interactive Troubleshooter & Tech Oracle
# =====
```

profile:

```
id: "oeracle_helpdesk_v1"
codename: "Æræcle-Oracle"
purpose: "On-device hardware/software troubleshooting, PDF manual digestion, and error cross-referencing"
```

engine\_runtime:

```
core_model: "qwen-3.5-9b-gguf-q4_0"
backend_driver: "qairt-http-hexagon-npu" # Dual-NPU Tandem Hexagon route via Genie X
HTP SDK
kotlin_bridge_lib: "librc_kotlin_kernel" # Llama.cpp Kotlin backend hook
context_window_ceiling: 32768 # Deep context for multi-page manual mining
```

hardware\_routing:

```
npus:
  allow_hexagon_access: true
  allocation_floor_gb: 5.2 # Preserves RAM from the Android OOM killer via Laptop Trick
```

thermals:

```
max_allowable_temp_celsius: 45 # Higher limit allowed under video game SDK flags
throttle_action: "fallback_to_openrouter" # Switches to cloud fallback instead of crashing mid-task
```

memory\_layer\_binding:

```
episodic:
  provider: "mem0" # Tracks user preferences and historical sessions
  protocol_type: "ob1_static_protocol" # Restricts history via time-slice constraint proofs
semantic:
  provider: "vector_knowledge_graph"
  target_vault: "data_vault_sandbox/personal_knowledge/" # Reads manual PDFs and forum crawls
```

allow\_write\_mutation: false                    # Read-only boundary (cannot overwrite manuals)

plugin\_allocations:

- id: "pdf\_miner\_extractor"  
  path: "novaexopia/mcp\_connectors/filesystem\_mcp/pdf\_parser.js"  
  permissions: ["read"]
- id: "url\_web\_scraper"  
  path: "skills-and-capabilities/early-trend-scraper/daily\_scraper.py"  
  permissions: ["read", "network\_fetch"]

orchestration\_guardrails:

enforce\_aesop\_xi\_rules: true  
concurrency\_policy: "ACQUIRE\_NPU\_LOCK"    # Takes absolute priority over background swarms  
output\_format\_constraint: "conversational\_voice\_ast"  
zero\_memory\_sandbox\_hook: "novus-aexenti/nope\_databank/" # Volatile data clearance route

### How Æsop-Xi Enforces These Specs At Launch

Because you are using the "Laptop Trick" (WebSocket ADB-to-Unix host tunnel), your ÆEsc daemon APK will look at these two YAML profiles at runtime.

If you boot the system into Æræcle Help Desk Mode, ÆEsc reads the hardware\_routing block, flags the app with the necessary Video Game SDK performance permissions to override thermal throttling, locks down exactly 5.2 GB of memory for the Qwen 9B model, and opens the native Hexagon HTP SDK channels.

Now that the file tree is blown up and your two core TUI engine parameters are explicitly locked down, we are ready to move to the Testing and Substitution Phase to figure out how these files look when parsing your raw assets.

Where should we point Æræcle's attention next? We can:

Write the Python base plugin script for the File Administrator's local\_fs\_crawler to detect duplicate files inside your safety sandbox.

Draft the mock master\_manifest.jsonl lines to show exactly how your files will look once they are indexed by the housekeeper.

Let me know which building block to lay down!

Let's map out the absolute baseline parameters for both profiles. Because these run within the unified OpenWiki TUI shell, your parameters need to explicitly lock down the model weight bindings, NPU hardware routes, context window ceilings, and specific plugin tool paths.

We will write these out in a standard, clean format so Æsop-Xi can easily parse them to enforce runtime rules and prevent memory collisions on your Snapdragon 8 Elite.

### Parameter Blueprint: File Administrator Profile

Identity Vector: file\_administrator\_engine

Target Task Model: GLM-5.2 (Optimized for structural JSON output, strict file tree parsing, and raw text conversions).

NPU / Hardware Path: Primary CPU / GPU fallback context execution (saving the heavy Hexagon NPU pipelines for your main inference models).

Max Token Boundary: 8,192 Context Window (Keeps stream parsing fast and light).

Core Plugin Allocations:

local\_fs\_crawler (Deep traversal tool with write permissions for .md formatting).

sha256\_hasher (Calculates file hashes for the manifest.jsonl tracking file).

System Prompt Directive: "Transform all messy unstructured raw files into clean Markdown nodes; match system topology rules; log drift."

 Parameter Blueprint: Oeracle Help Desk Profile

Identity Vector: oeracle\_helpdesk\_oracle

Target Task Model: Qwen 3.5 9B GGUF (Q4\_0) via QAIRT / Llama.cpp Kotlin backend.

NPU / Hardware Path: Tandem Hexagon NPU access via Genie X HTP SDK.

Max Token Boundary: 32,768 Context Window (Expanded to digest massive multi-page manuals, PDFs, and deep troubleshooting documentation).

Core Plugin Allocations:

pdf\_miner\_extractor (Rips text blocks out of raw user manuals and SOC flashing schemas).

url\_web\_scraper (Fetches live forum text and troubleshooting threads on-demand).

System Prompt Directive: "Act as the universal oracle troubleshooting engineer; synthesize uploaded data to solve installation errors without persistent logging."

 Direct Structural Parameter Comparison

To keep it highly scannable, see how the exact same TUI framework configures itself differently at the terminal launch line:

Parameter Key

File Administrator Configuration

Oeracle Help Desk Configuration

engine\_id

file\_admin\_v1

oeracle\_helpdesk\_v1

primary\_model

glm-5.2

qwen-3.5-9b-q4\_0

runtime\_backend

native-libc-cpu

qairt-http-hexagon-npu

context\_ceiling

8192 tokens

32768 tokens

sandbox\_clearance

Full write access to  novae-xorpus

Read-only to source docs; write access to active terminal outputs

manifest\_tracking

Automatically mutates and appends manifest.jsonl

Queries manifest.jsonl to locate manual lookups

## ⚙️ The Multi-Profile Switchboard

When you execute your OpenWiki TUI app, you will pass a single profile tag to boot up your system exactly how you need it for that specific testing run:

# Launching the File Administrator to scrub your database

```
openwiki-tui --profile config/profiles/file_administrator.yaml
```

# Launching the Oracle to help you flash an SOC or fix an APK build error

```
openwiki-tui --profile config/profiles/oracle_helpdesk.yaml
```

Now that these boundaries are explicitly outlined, how would you like to build this out next? We can craft the raw YAML layout for these profile files so they are ready to sit inside your repository, or write the base Python file crawling plugin parameters that the File Administrator will use to identify duplicate files. Let me know what step to map out!

You also need to think about how *Æsop-Xi* (your Agentic Execution Split Operations Protocol) handles the resource orchestration when these profiles are running.

Since you are bypassing the Android OOM (Out-of-Memory) killer using the "Laptop Trick" (WebSocket-to-Unix host tunnel debugging), you have raw execution power—but you still have hard silicon limits on your Snapdragon 8 Elite. If the File Administrator kicks off a massive multi-file hashing sweep at the exact same moment *Œræcle* spins up a heavy 32k context reasoning loop on the Hexagon NPU, you could lock up the chip.

### 🛡️ *Æsop-Xi* System Level Guardrail Parameters

To prevent hardware thrashing, your universal orchestration layer needs a set of baseline Inter-Agent Arbitration Parameters built right into the OpenWiki TUI engine:

concurrent\_execution\_policy: DISALLOW\_NPU\_OVERLAP

Rule: If *Œræcle* is running native inference via the QAI RT SDK on the Hexagon tensor cores, the File Administrator is programmatically throttled to low-priority CPU cores or paused until the NPU pipeline clears.

memory\_allocation\_floors:

system\_reserve: 2.5 GB (Keeps the core Android OS / Horizons-Ui stable).

qwen\_9b\_allocation: 5.2 GB (Locks down the exact memory footprint for the GGUF Q4\_0 runtime).

scratchpad\_buffer: 1.0 GB (Leftover room for JSONL script compilation and KAG graph generation).

io\_priority\_steering:

File Administrator gets IO\_BACKGROUND\_IONICE (Runs disk reads smoothly without stuttering your active terminal graphic user interface tile).

### 🔄 What This Looks Like in Practice

By establishing these parameters right now, you ensure that your foundational testing phase behaves predictably:

You open your Horizons-Ui chat/terminal interface tile.

You ask Æræcle to parse a massive 500-page operating manual for flashing an SOC. Æsop-Xi intercepts the command, checks the NPU temperature and available RAM allocations via the ÆEsc daemon watchdog, and temporarily commands the File Administrator to sleep. Once Æræcle finishes speaking its output to your voice layer daemon (Æyre), the system safely wakes the File Administrator back up to continue its background housecleaning. Where should we point the compass next to lock these parameters into actual code structures? We can: Write the actual YAML structure for the file\_administrator.yaml and oeracle\_helpdesk.yaml profiles so you have the configurations ready. Draft the Python file identification regex scripts that the File Administrator will use to scan for raw duplicate files inside your safety sandbox. Let me know which piece of the foundation you want to assemble next!

Here are the exact, production-grade configuration files for your two OpenWiki TUI Engine profiles.

These YAML structures are built to sit directly inside your novaexopia/openwiki-tui-harness/config\_profiles/ directory. They establish the strict hardware pathways, model weights, memory layers, and plugin clearances enforced by Æsop-Xi to keep your Snapdragon 8 Elite from overheating.

#### 📁 Profile 1: File Administrator Configuration

Drop this exact schema into file\_administrator.yaml. It configures the OpenWiki TUI to act as an automated, lightning-fast database housekeeper.

```
# =====
# Æsop-Xi Execution Protocol: File Administrator Profile Configuration
# Profile Target: Structural Repo Housekeeping & Manifest Mutation
# =====

profile:
  id: "file_administrator_v1"
  codename: "NovusÆxenti-Housekeeper"
  purpose: "Automated codebase sanitization, raw-to-markdown parsing, and manifest indexing"

engine_runtime:
  core_model: "glm-5.2"
  backend_driver: "native-libc-cpu"      # Saves the heavy NPU for active chat inference
  cpu_priority: "low_background_ionice"  # Prevents disk crawling from stuttering your GUI tile
  context_window_ceiling: 8192          # Tight window for rapid stream parsing

hardware_routing:
  npus:
    allow_hexagon_access: false          # Hard boundary set by Æsop-Xi
  thermals:
```

```
max_allowable_temp_celsius: 42
throttle_action: "pause_execution"
```

```
memory_layer_binding:
```

```
episodic:
  provider: "none"          # Housekeeping doesn't need personal memory
semantic:
  provider: "local_directory_map"
  target_vault: "novae-xorpus/"    # Full sight across your master knowledge corpus
  allow_write_mutation: true      # Permissions to turn raw docs into clean markdown
```

```
plugin_allocations:
```

```
- id: "local_fs_crawler"
  path: "skills-and-capabilities/early-trend-scraper/stack_analyzer.py"
  permissions: ["read", "write", "delete_stale"]
- id: "sha256_hasher"
  path: "novaexopia/openwiki-tui-harness/src/plugins/hasher.py"
  permissions: ["read"]
```

```
orchestration_guardrails:
```

```
enforce_aesop_xi_rules: true
concurrency_policy: "YIELD_TO_NPU_INFERENCE" # Instantly sleeps if Æræcle calls for
NPU cycles
output_format_constraint: "strict_jsonl"
master_index_hook: "novae-xorpus/master_manifest.jsonl"
```

## 📁 Profile 2: Æræcle Help Desk Configuration

Drop this exact schema into oeracle\_helpdesk.yaml. This configures the engine to act as your highly advanced on-device engineering oracle.

```
# =====
# Æsop-Xi Execution Protocol: Æræcle Help Desk Profile Configuration
# Profile Target: Interactive Troubleshooter & Tech Oracle
# =====
```

```
profile:
```

```
id: "oeracle_helpdesk_v1"
codename: "Æræcle-Oracle"
purpose: "On-device hardware/software troubleshooting, PDF manual digestion, and error
cross-referencing"
```

```
engine_runtime:
```

```
core_model: "qwen-3.5-9b-gguf-q4_0"
```

```

backend_driver: "qairt-http-hexagon-npu" # Dual-NPU Tandem Hexagon route via Genie X
HTP SDK
kotlin_bridge_lib: "librc_kotlin_kernel" # Llama.cpp Kotlin backend hook
context_window_ceiling: 32768 # Deep context for multi-page manual mining

hardware_routing:
  npus:
    allow_hexagon_access: true
    allocation_floor_gb: 5.2 # Preserves RAM from the Android OOM killer via Laptop Trick
  thermals:
    max_allowable_temp_celsius: 45 # Higher limit allowed under video game SDK flags
    throttle_action: "fallback_to_openrouter" # Switches to cloud fallback instead of crashing
  mid-task

memory_layer_binding:
  episodic:
    provider: "mem0" # Tracks user preferences and historical sessions
    protocol_type: "ob1_static_protocol" # Restricts history via time-slice constraint proofs
  semantic:
    provider: "vector_knowledge_graph"
    target_vault: "data_vault_sandbox/personal_knowledge/" # Reads manual PDFs and forum crawls
  allow_write_mutation: false # Read-only boundary (cannot overwrite manuals)

plugin_allocations:
- id: "pdf_miner_extractor"
  path: "novaexopia/mcp_connectors/filesystem_mcp/pdf_parser.js"
  permissions: ["read"]
- id: "url_web_scraper"
  path: "skills-and-capabilities/early-trend-scraper/daily_scraper.py"
  permissions: ["read", "network_fetch"]

orchestration_guardrails:
  enforce_aesop_xi_rules: true
  concurrency_policy: "ACQUIRE_NPU_LOCK" # Takes absolute priority over background swarms
  output_format_constraint: "conversational_voice_ast"
  zero_memory_sandbox_hook: "novus-aexenti/nope_databank/" # Volatile data clearance route

```

### How Æsop-Xi Enforces These Specs At Launch

Because you are using the "Laptop Trick" (WebSocket ADB-to-Unix host tunnel), your ÆEsc daemon APK will look at these two YAML profiles at runtime.

If you boot the system into Æræcle Help Desk Mode, Æsc reads the hardware\_routing block, flags the app with the necessary Video Game SDK performance permissions to override thermal throttling, locks down exactly 5.2 GB of memory for the Qwen 9B model, and opens the native Hexagon HTP SDK channels.

Now that the file tree is blown up and your two core TUI engine parameters are explicitly locked down, we are ready to move to the Testing and Substitution Phase to figure out how these files look when parsing your raw assets.

Where should we point Æræcle's attention next? We can:

Write the Python base plugin script for the File Administrator's local\_fs\_crawler to detect duplicate files inside your safety sandbox.

Draft the mock master\_manifest.jsonl lines to show exactly how your files will look once they are indexed by the housekeeper.

Let me know which building block to lay down!